

Agenda

- ArrayList
- Queue
- LinkedList
- ~~Set~~
- ~~Map~~
- ~~hashCode()~~

ArrayList class

- Internally ArrayList is dynamic array (can grow or shrink dynamically).
- Elements can be traversed using Iterator, ListIterator, or using index.
- Primary uses
 - Random access
 - Add/remove elements (at the end)
- Limitations
 - Slower add/remove in between the collection
 - Uses more contiguous memory
- Inherited from List<>.

LinkedList class

- Internally LinkedList is doubly linked list.
- Elements can be traversed using Iterator, ListIterator, or using index.
- Primary use
 - Add/remove elements (anywhere)
 - Less contiguous memory available
- Limitations:
 - Slower random access
- Inherited from List<>, Deque<>.

Queue Interface

- Represents utility data structures (like Stack, Queue, ...) data structure.
- Implementations: LinkedList, ArrayDeque, PriorityQueue.
- Can be accessed using iterator, but no random access.
- Methods
 - boolean add(E e) - throw IllegalStateException if full.
 - E remove() - throw NoSuchElementException if empty
 - E element() - throw NoSuchElementException if empty
 - boolean offer(E e) - return false if full.
 - E poll() - returns null if empty
 - E peek() - returns null if empty
- In queue, addition and deletion is done from the different ends (rear and front).
- Difference between these methods is first 3 methods throws exception however next 3 methods do not throw exception if operation fails.

PriorityQueue class

- Internally PriorityQueue is a "binary heap" (Array implementation of binary Tree) data structure.
- Elements with highest priority is deleted first (NOT FIFO).
- Elements should have natural ordering or need to provide comparator.

Assignments

- Q1. Look for the Deque Interface and understand the ArrayDeque class
- Q2. Use LinkedList class to have the implementation of
 1. List
 2. Queue
 3. Stack

SUNBEAM